

Part D — Data Structures & Algorithms

Interview-complete edition: recursion, backtracking, tries, greedy, and bit manipulation added; sorting and hashing reordered; every core topic expanded with the gaps an interview loop actually tests.

Phase	9
Topics	16 core + 1 optional/advanced
Estimated Duration	10–13 weeks
Difficulty	Intermediate → Advanced
Suggested Pace	8–12 hrs/week · 2–3 topics/week (part-time default)
Projects Included	5–7 detailed hands-on projects per topic, step-by-step (85 total)
LeetCode Volume	~163 problems distributed across topics, ~65% Medium
Documentation Sources	2–3 official/primary sources per topic

What Changed From the Original Part D

- Added 5 new topics: Recursion (taught first, not absorbed by osmosis), Backtracking, Tries (promoted out of a footnote), Greedy Algorithms, and Bit Manipulation.
- Reordered: Sorting & Searching now comes before Trees/Graphs/DP, since binary search is a prerequisite mental model, not just its own topic. Hashing moved up to sit right after Arrays.
- Elevated Union-Find from a sub-item of the Graphs topic into its own short, focused topic.
- Augmented existing topics: Kadane's Algorithm (Arrays), monotonic deque + min-stack (Stacks & Queues), LRU Cache as a full project (Linked Lists), a BFS shortest-path bridge project before Dijkstra (Graphs), House Robber + State-Machine DP (Dynamic Programming).
- Added a suggested LeetCode problem count per topic (~163 total, ~65% Medium) — the from-scratch builds teach mechanism, LeetCode volume teaches pattern recognition under time pressure. Neither replaces the other.
- Segment Trees / Fenwick Trees added as an explicitly optional appendix topic (9.16) — correctly out of the core loop unless you're targeting quant/HFT or competitive-programming-heavy interviews.

Recursion & Recursive Thinking

The mental model behind Trees, Backtracking, and DP — taught first, not absorbed by accident.

Subtopics: base case & recursive case · the call stack, visualized · recursion trees · converting recursion to iteration · tail vs. non-tail recursion

Suggested Projects — Detailed Instructions

1. Factorial & Fast Power — the simplest base/recursive-case pair

Steps: implement factorial(n) recursively → implement power(base, exp) using fast exponentiation (halve exp each call) → trace the call stack on paper for $n = 5$ → convert both to iterative versions and compare space complexity

2. Sum of Digits & Digit Reversal — recursion on numbers, not collections

Steps: write a recursive sumOfDigits(n) → write a recursive reverseNumber(n) → test both with negative numbers and single-digit input → identify exactly which base case prevents infinite recursion

3. Recursive Array Search & Sum — recursion without built-ins

Steps: implement a recursive linearSearch(arr, target, index) → implement a recursive arraySum(arr, index) → implement a recursive arrayMax(arr, index) → test each on an empty array and a single-element array

4. Tower of Hanoi — the problem that teaches recursion to itself

Steps: implement the 3-peg recursive solution and print each move → count total moves for n disks and confirm it equals $2^n - 1$ → explain in a comment why this is awkward to do iteratively → time it for $n = 20$ and discuss the growth rate

5. Recursion → Iteration Conversion Drill

Steps: take 3 recursive solutions from this topic → convert each to iterative using an explicit stack where needed → compare space complexity before and after → note which direction felt natural and which felt forced, and why

Study Resources

- GeeksforGeeks — Recursion in Java — <https://www.geeksforgeeks.org/java/recursion-in-java/>
- GeeksforGeeks — Recursive Algorithms — <https://www.geeksforgeeks.org/dsa/recursion-algorithms/>
- NeetCode — Roadmap — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 8 problems (mostly Easy/Medium), on top of the hands-on builds above.

Arrays & Strings

The most fundamental structure, and the pattern toolkit that solves a huge share of interview questions.

Subtopics: two pointers · sliding window · prefix sums · Kadane's algorithm (max subarray) · StringBuilder & string manipulation

Suggested Projects — Detailed Instructions

1. Two Sum (and variants) — the canonical two-pointer/hash problem

Steps: solve it brute-force $O(n^2)$ first → solve it again with a HashMap in $O(n)$ → solve the sorted-array version with two pointers → compare all three approaches' time complexity

2. Longest Substring Without Repeating Characters — the sliding window pattern

Steps: use a window with two pointers and a Set → track the max length as the window grows/shrinks → test with edge cases: empty string, all-same characters

3. Prefix Sum Range Queries — $O(1)$ range sums after $O(n)$ setup

Steps: build a prefix sum array from an input array → answer 'sum from index i to j ' queries in $O(1)$ → compare to recomputing the sum each time → apply it to a 'subarray sums to target' problem

4. Kadane's Algorithm — Maximum Subarray

Steps: track a running sum, resetting it when it drops below the current element → track the running max separately from the running sum → extend it to also return the subarray's start/end indices → test with an all-negative array

5. String Reversal & Palindrome Check

Steps: reverse a string with StringBuilder, then again with two pointers in-place → check if a string is a palindrome using two pointers → handle spaces/punctuation/case in the palindrome check

6. Merge Intervals — a very common array-of-pairs pattern

Steps: sort intervals by start time → merge overlapping intervals in one pass → return the minimal merged list → test with intervals fully contained within others

Study Resources

- GeeksforGeeks — Array Data Structure — <https://www.geeksforgeeks.org/dsa/array-data-structure-guide/>
- NeetCode — Arrays & Hashing — <https://neetcode.io/roadmap>
- Oracle — Arrays (Java Tutorials) — <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>

Suggested LeetCode volume: 15 problems (5 Easy / 8 Medium / 2 Hard), on top of the hands-on builds above.

Hashing

Trading memory for near-constant-time lookups — moved up to sit right after Arrays, where it's used constantly.

Subtopics: HashMap internals (buckets & hashing) · collision handling · frequency maps · the two-sum family

Suggested Projects — Detailed Instructions

1. Implement a Basic HashMap

Steps: back it with an array of buckets → implement hashCode-based bucket selection → handle collisions with chaining (a linked list per bucket) → test with intentionally colliding keys

2. Frequency Map Drill

Steps: count character frequency in a string → count word frequency in a sentence → find the most frequent element in an array → solve 'first non-repeating character' with the same technique

3. Two Sum Family, Revisited

Steps: solve two-sum with a HashMap in one pass → solve three-sum by fixing one element and hashing the rest → compare time complexity to the brute-force versions

4. Group Anagrams — hashing with a computed key

Steps: group a list of words into anagram sets → use the sorted-characters string as the hash key → return groups as a List<List<String>> → test with words that share no anagrams

5. Longest Consecutive Sequence — $O(n)$ with a HashSet, not sorting

Steps: put all numbers into a HashSet → for each number, check if it's the start of a sequence → extend forward counting consecutive numbers → confirm this beats the $O(n \log n)$ sort-based approach

Study Resources

- GeeksforGeeks — Hashing Data Structure — <https://www.geeksforgeeks.org/dsa/hashing-data-structure/>
- Oracle — HashMap (Javadoc) — <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html>
- NeetCode — Arrays & Hashing — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 10 problems (4 Easy / 5 Medium / 1 Hard), on top of the hands-on builds above.

Two Pointers & Sliding Window — Pattern Review

Pulled out of Arrays into its own explicit review so the pattern gets named, not just used once and forgotten.

Subtopics: fixed vs. variable window · shrink/grow conditions · monotonic deque (sliding window maximum) · when two pointers beats a window

Suggested Projects — Detailed Instructions

1. Sliding Window Maximum — the optimal $O(n)$ version

Steps: maintain a monotonic decreasing deque of indices → pop from the back while the new element is larger → pop from the front once the front index leaves the window → return the front of the deque as the max at each step

2. Minimum Window Substring — the hardest common variable-window problem

Steps: expand the window until it contains all required characters → shrink from the left while it's still valid, tracking the minimum → use a frequency map to know when the window is 'valid' → test with a target string longer than any valid window

3. Fixed-Size Window Drill

Steps: solve 'max sum subarray of size k' → solve 'first negative number in every window of size k' → compare a naive $O(n*k)$ re-scan approach to the $O(n)$ sliding version

4. Two Pointers vs. Sliding Window — when each applies

Steps: solve 'container with most water' with two pointers → solve 'longest substring with at most K distinct characters' with a window → write a short comparison: which signal in a problem statement tells you which pattern to reach for

5. Find the Duplicate Number — fast/slow pointers on an array, not a list

Steps: treat array values as a linked-list's 'next' pointers → apply Floyd's cycle detection to find the duplicate → confirm it works without modifying the array or using extra space

Study Resources

- GeeksforGeeks — Sliding Window Technique — <https://www.geeksforgeeks.org/dsa/window-sliding-technique/>
- NeetCode — Sliding Window — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 8 problems (3 Easy / 5 Medium), on top of the hands-on builds above.

Stacks & Queues

LIFO and FIFO structures underlying everything from undo buttons to breadth-first search.

Subtopics: stack (push/pop/peek) · queue (enqueue/dequeue) · min-stack · monotonic stack · implementing both from scratch

Suggested Projects — Detailed Instructions

1. Stack From Scratch — array-backed, no `java.util.Stack`

Steps: implement push/pop/peek/isEmpty backed by a resizable array → handle the empty-pop case with a clear exception → test push/pop order matches LIFO expectations

2. Valid Parentheses — the classic stack interview question

Steps: use a stack to match opening/closing brackets → handle all 3 bracket types at once: `()[]{} → return false` on any mismatch or leftover open bracket → test with nested and malformed inputs

3. Min-Stack — $O(1)$ `getMin()` alongside push/pop

Steps: maintain a second stack tracking the running minimum → push onto the min-stack only when a new minimum arrives (or track it per-frame) → pop from both stacks together → test with duplicate minimums and after popping the current minimum off

4. Queue From Scratch — array or linked-list backed

Steps: implement enqueue/dequeue/peek/isEmpty → decide array vs. linked-list backing and justify it in a comment → test FIFO order is correct

5. Monotonic Stack: Next Greater Element

Steps: for each element, find the next element to its right that's greater → solve it in $O(n)$ using a monotonic decreasing stack → compare to the brute-force $O(n^2)$ approach → test with a strictly decreasing input (no next-greater exists)

6. Implement a Queue Using Two Stacks

Steps: implement enqueue/dequeue using only two Stack instances internally → figure out which operations need to move elements between stacks → test the amortized cost stays reasonable

Study Resources

- GeeksforGeeks — Stack Data Structure — <https://www.geeksforgeeks.org/dsa/stack-data-structure/>
- GeeksforGeeks — Queue Data Structure — <https://www.geeksforgeeks.org/dsa/queue-data-structure/>
- NeetCode — Stack — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 10 problems (3 Easy / 6 Medium / 1 Hard), on top of the hands-on builds above.

Sorting & Searching

Moved earlier — binary search variants are a prerequisite mental model for the Graph and DP work ahead, not just an interview topic on their own.

Subtopics: merge sort · quick sort · binary search & its variants · time complexity trade-offs

Suggested Projects — Detailed Instructions

1. Merge Sort From Scratch — a stable $O(n \log n)$ sort

Steps: implement the recursive divide step → implement the merge step combining two sorted halves → test on an already-sorted and a reverse-sorted array → confirm it's stable (equal elements keep relative order)

2. Quick Sort From Scratch — an in-place $O(n \log n)$ average-case sort

Steps: implement the partition step (Lomuto or Hoare) → implement the recursive quicksort around it → test worst-case behavior on an already-sorted array → discuss why pivot choice matters, in a comment

3. Binary Search & Its Variants

Steps: implement standard binary search for an exact match → implement 'find first occurrence' in a sorted array with duplicates → implement 'find insertion point' for a value not present → test all three against edge cases: empty array, single element

4. Search in a Rotated Sorted Array

Steps: given a sorted array rotated at an unknown pivot, find a target in $O(\log n)$ → figure out which half is properly sorted at each step → test with the target being the pivot element itself

5. Sorting Algorithm Comparison

Steps: time your merge sort and quick sort on the same large random array → time Java's built-in `Arrays.sort()` on the same data → explain any difference you observe → note which algorithm you'd pick for nearly-sorted data

Study Resources

- GeeksforGeeks — Sorting Algorithms — <https://www.geeksforgeeks.org/dsa/sorting-algorithms/>
- GeeksforGeeks — Binary Search — <https://www.geeksforgeeks.org/dsa/binary-search/>
- NeetCode — Binary Search — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 10 problems (3 Easy / 6 Medium / 1 Hard), on top of the hands-on builds above.

Linked Lists

A chain of nodes where insertion/deletion is cheap but random access isn't — the classic source of pointer-manipulation questions.

Subtopics: singly & doubly linked lists · fast/slow pointer technique · reversing a list · cycle detection

Suggested Projects — Detailed Instructions

1. Build a Singly Linked List From Scratch

Steps: write a Node class with value and next → implement add, remove, and get(index) → implement a toString() for easy debugging → test insertion at head, middle, and tail

2. Reverse a Linked List — the single most common linked-list question

Steps: reverse it iteratively with 3 pointers (prev/curr/next) → reverse it again recursively → compare both approaches' space complexity → test on an empty list and a 1-node list

3. Detect a Cycle (Floyd's Algorithm)

Steps: build a list with a deliberate cycle → implement fast/slow pointers to detect it → extend it to find where the cycle begins → test on a list with no cycle to confirm no false positive

4. Doubly Linked List Implementation

Steps: extend your Node with a prev reference → implement insertion/removal from both ends in $O(1)$ → implement forward and backward traversal → use this as the base for this topic's LRU Cache project

5. Merge Two Sorted Linked Lists

Steps: merge two already-sorted linked lists into one sorted list → do it without creating new nodes, just relinking → test with lists of different lengths → test with one empty input list

6. LRU Cache — the payoff project for building a doubly linked list

Steps: combine a HashMap ($O(1)$ lookup) with your doubly linked list ($O(1)$ reorder) → implement get(key) moving the accessed node to the front → implement put(key, value) evicting the tail node when over capacity → test eviction order explicitly with a capacity of 2

Study Resources

- GeeksforGeeks — Linked List Data Structure — <https://www.geeksforgeeks.org/dsa/linked-list-data-structure/>
- Baeldung — Introduction to Linked Lists in Java — <https://www.baeldung.com/java-linkedlist>
- NeetCode — Linked List — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 10 problems (3 Easy / 6 Medium / 1 Hard), on top of the hands-on builds above.

Backtracking

One of the highest-yield patterns for new-grad/internship loops specifically — and it didn't exist anywhere in the original curriculum.

Subtopics: the choose/explore/un-choose template · pruning · subsets vs. permutations vs. combinations · constraint satisfaction

Suggested Projects — Detailed Instructions

1. Subsets — the template project every other backtracking problem builds on

Steps: write the choose / explore / un-choose recursive template → generate all subsets of a set with no duplicates → extend it to handle duplicate elements without duplicate subsets → confirm the count matches 2^n

2. Permutations

Steps: generate all permutations of a list using swap-based backtracking → generate them again using a 'used[]' boolean array approach → compare the two implementations' clarity and space use → handle duplicate elements without duplicate permutations

3. Combination Sum

Steps: find all combinations that sum to a target, reusing numbers → add a variant where each number can be used at most once → prune branches early once the running sum exceeds the target → test with a target that has no valid combination

4. N-Queens

Steps: place queens row by row, checking column/diagonal conflicts → use sets (or bitmasks) for $O(1)$ conflict checking → count total solutions for $n = 8$ → print one valid board as a sanity check

5. Word Search

Steps: DFS from each cell trying to match the next character → mark cells visited during the current path and un-mark on backtrack → prune as soon as a path can't match → test with a word that wraps around the edge of the grid

Study Resources

- GeeksforGeeks — Backtracking Algorithms — <https://www.geeksforgeeks.org/dsa/backtracking-algorithms/>
- GeeksforGeeks — Introduction to Backtracking — <https://www.geeksforgeeks.org/dsa/introduction-to-backtracking-2/>
- NeetCode — Backtracking — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 10 problems (1 Easy / 7 Medium / 2 Hard), on top of the hands-on builds above.

Trees

Hierarchical, recursive structures — the shape behind file systems, category menus, and JSON itself.

Subtopics: binary tree traversals (in/pre/post/BFS) · binary search tree operations · tree height & balance · lowest common ancestor

Suggested Projects — Detailed Instructions

1. Binary Tree Traversals

Steps: implement in-order traversal recursively → implement pre-order and post-order recursively → implement level-order (BFS) using a Queue → print all four for the same tree and compare output

2. Binary Search Tree From Scratch

Steps: implement insert() maintaining BST ordering → implement search() in $O(\log n)$ on a balanced tree → implement delete() handling all 3 cases: leaf, one child, two children → implement an in-order iterator confirming sorted output

3. Tree Height & Balance Check

Steps: write a recursive height(node) function → write isBalanced() checking every subtree's height difference → test on a balanced tree and a deliberately skewed one

4. Lowest Common Ancestor

Steps: implement LCA for a binary search tree using ordering properties → implement it again for a general binary tree with no ordering to rely on → compare the two approaches → test with one node being an ancestor of the other

5. Serialize & Deserialize a Binary Tree

Steps: design a string format encoding the tree, including nulls → implement serialize() producing that string → implement deserialize() rebuilding the exact same tree → confirm round-tripping produces an identical tree

Study Resources

- GeeksforGeeks — Tree Data Structure — <https://www.geeksforgeeks.org/dsa/tree-data-structure/>
- Baeldung — Binary Tree in Java — <https://www.baeldung.com/java-binary-tree>
- NeetCode — Trees — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 15 problems (4 Easy / 9 Medium / 2 Hard), on top of the hands-on builds above.

Heaps & Priority Queues

A tree-shaped structure optimized for one thing: instantly grabbing the smallest (or largest) item.

Subtopics: min-heap vs. max-heap · heapify (sift up/down) · top-K problems · heap sort

Suggested Projects — Detailed Instructions

1. Min-Heap From Scratch — array-backed, no PriorityQueue built-in

Steps: implement insert() with sift-up → implement extractMin() with sift-down → implement peek() → test the heap property holds after several insert/extract cycles

2. Kth Largest Element — the signature top-K heap problem

Steps: maintain a min-heap of size K while scanning the input → return the heap's root as the Kth largest at the end → compare this to fully sorting the array first → test with K equal to the array length

3. Heap Sort — an $O(n \log n)$ sort using your own heap

Steps: build a max-heap from an unsorted array → repeatedly extract the max and place it at the end → confirm the result is fully sorted → compare its behavior to Java's built-in sort on the same input

4. Merge K Sorted Lists — a classic heap application

Steps: put the head of each list into a min-heap → repeatedly extract the min and advance that list → build the merged result → test with lists of very different lengths

5. Running Median with Two Heaps

Steps: maintain a max-heap for the lower half and a min-heap for the upper half → keep them balanced as numbers stream in → return the median in $O(1)$ at any point → test with an even and an odd number of elements seen so far

6. Top-K & Scheduling Patterns — naming the two-heap/priority-queue pattern explicitly

Steps: solve 'K Closest Points to Origin' with a max-heap of size K → solve 'Task Scheduler' using a max-heap on task frequency plus a cooldown queue → identify what both problems have in common: bounding a heap's size to control complexity

Study Resources

- GeeksforGeeks — Heap Data Structure — <https://www.geeksforgeeks.org/dsa/heap-data-structure/>
- Oracle — PriorityQueue (Javadoc) — <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/PriorityQueue.html>
- NeetCode — Heap / Priority Queue — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 8 problems (2 Easy / 5 Medium / 1 Hard), on top of the hands-on builds above.

Tries

Promoted from a footnote in the original final project — it was referenced ('trie-based autocomplete') but never actually taught.

Subtopics: TrieNode structure · insert/search/startsWith · deletion · word-frequency & autocomplete ranking

Suggested Projects — Detailed Instructions

1. Trie From Scratch

Steps: write a TrieNode with a children map/array and an isEndOfWord flag → implement insert(word) → implement search(word) for exact matches → implement startsWith(prefix) for prefix matches

2. Delete From a Trie

Steps: implement delete(word), handling the case where the word is a prefix of another → prune nodes that become childless after deletion → test deleting a word that shares a prefix with another stored word

3. Word Search II — Trie + Backtracking combined

Steps: build a Trie from the dictionary of words to find → DFS the grid, walking the Trie alongside the current path → prune a DFS branch as soon as the Trie has no matching child → test with overlapping words that share prefixes

4. Autocomplete System

Steps: store a frequency/weight at each end-of-word node → given a prefix, collect all completions under that Trie node → sort completions by frequency, then alphabetically → test with a prefix that matches zero completions

5. Longest Common Prefix — a simpler Trie warm-up, done two ways

Steps: solve it by inserting all words into a Trie and walking down while there's exactly one child → solve it again with simple horizontal string scanning, no Trie → compare the two approaches for a large list of short words vs. a short list of long words

Study Resources

- GeeksforGeeks — Trie Data Structure in Java — <https://www.geeksforgeeks.org/java/trie-data-structure-in-java/>
- Baeldung — Trie Data Structure in Java — <https://www.baeldung.com/trie-java>
- NeetCode — Tries — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 5 problems (1 Easy / 3 Medium / 1 Hard), on top of the hands-on builds above.

Graphs

Nodes and edges modeling relationships — from social networks to delivery routes to the N+1 query problem from Part A.

Subtopics: adjacency list vs. matrix · BFS & DFS · unweighted shortest path (BFS) · topological sort · Dijkstra's algorithm · union-find

Suggested Projects — Detailed Instructions

1. Graph Representation & Traversal

Steps: build a graph using an adjacency list: `Map<Node, List<Node>>` → implement BFS returning visit order → implement DFS both iteratively (with a stack) and recursively → test on a graph with a disconnected component

2. Connected Components

Steps: given an undirected graph, count the number of connected components → use DFS or union-find to group nodes → test on a graph that's fully connected vs. fully disconnected

3. Unweighted Shortest Path — BFS as the bridge to Dijkstra

Steps: use BFS to find the shortest path (fewest edges) from a source in an unweighted graph → track distances in a visited/distance map as you go → reconstruct the actual path, not just the distance → explain in a comment exactly why BFS fails once edges have weights — this motivates Dijkstra next

4. Topological Sort

Steps: model a set of tasks with prerequisites as a directed graph → implement topological sort via DFS or Kahn's algorithm (BFS + in-degree) → detect and report a cycle (an impossible ordering) if one exists

5. Dijkstra's Shortest Path — weighted graph pathfinding

Steps: model a weighted graph, e.g. a delivery route network → implement Dijkstra using a priority queue → return the shortest distance to every node from a source → test against a graph where the 'obvious' path isn't shortest

6. Union-Find as an Alternative to DFS

Steps: implement `union()` and `find()` with path compression → use it to detect a cycle in an undirected graph → use it to count connected components again and compare to your DFS version from project 2

Study Resources

- GeeksforGeeks — Graph Data Structure and Algorithms — <https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/>
- Baeldung — Graph Theory in Java — <https://www.baeldung.com/java-graphs>
- NeetCode — Graphs — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 15 problems (3 Easy / 9 Medium / 3 Hard), on top of the hands-on builds above.

Union-Find (Disjoint Set)

Elevated from a sub-item of the Graphs topic to its own short, focused pattern family.

Subtopics: parent array representation · find() with path compression · union() by rank/size · when Union-Find beats DFS

Suggested Projects — Detailed Instructions

1. Union-Find From Scratch

Steps: implement find() with path compression → implement union() by rank or size → test that two elements unioned indirectly (through a chain) still resolve to the same root

2. Number of Provinces / Friend Circles

Steps: union every pair of directly-connected nodes → count the number of distinct roots remaining → compare runtime to a DFS-based connected-components solution

3. Redundant Connection

Steps: process edges one at a time, unioning each pair → the first edge that unions two nodes already in the same set is the redundant one → return that edge → test with a graph that has no redundant edge

4. Accounts Merge — Union-Find with a real-world flavor

Steps: union accounts that share at least one email → group all emails by their final root account → test with accounts that chain together transitively (A-B, B-C, so A-C are merged)

Study Resources

- GeeksforGeeks — Disjoint Set (Union-Find) — <https://www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/>
- NeetCode — Union-Find / Graphs — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 5 problems (1 Easy / 3 Medium / 1 Hard), on top of the hands-on builds above.

Greedy Algorithms

The conceptual counterweight to DP — teaching DP without ever contrasting it against Greedy leaves a real gap in interview discussions.

Subtopics: the greedy-choice property · optimal substructure · interval scheduling · greedy vs. DP: how to tell which one applies

Suggested Projects — Detailed Instructions

1. Activity/Interval Selection

Steps: sort activities by end time → greedily pick the next activity that starts after the last one picked ends → prove to yourself (in a comment) why sorting by end time, not start time, is what makes this greedy choice correct

2. Jump Game

Steps: track the farthest index reachable so far while scanning left to right → return false as soon as the current index exceeds the farthest reachable → extend it to Jump Game II: minimum number of jumps to reach the end

3. Gas Station

Steps: track a running tank total across the whole loop once → if the running total ever goes negative, the start candidate resets to the next station → prove why one linear pass is sufficient instead of checking every starting station

4. Greedy vs. DP: Where Greedy Fails

Steps: solve 0/1 Knapsack with a naive greedy (highest value/weight ratio first) → construct a test case where that greedy answer is wrong → explain in a comment exactly which property (greedy-choice or optimal substructure) breaks for 0/1 Knapsack but holds for Fractional Knapsack

5. Huffman Coding (conceptual build)

Steps: build a frequency map of characters in a string → repeatedly merge the two lowest-frequency nodes using a min-heap → derive the resulting prefix codes from the built tree → confirm no code is a prefix of another

Study Resources

- GeeksforGeeks — Greedy Algorithms — <https://www.geeksforgeeks.org/dsa/greedy-algorithms/>
- GeeksforGeeks — Introduction to Greedy Algorithms — <https://www.geeksforgeeks.org/dsa/introduction-to-greedy-algorithm-data-structures-and-algorithm-tutorials/>
- NeetCode — Greedy — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 8 problems (3 Easy / 4 Medium / 1 Hard), on top of the hands-on builds above.

Bit Manipulation

Low time investment, disproportionately common as an OA screen and 'easy warm-up' interview question.

Subtopics: AND/OR/XOR/NOT & shifts · counting set bits · checking powers of two · bitmasking basics

Suggested Projects — Detailed Instructions

1. Bitwise Operator Drill

Steps: write helper methods for setBit, clearBit, toggleBit, and checkBit at a given position → confirm each against a manually-worked binary example → explain in a comment why $n \& (n-1)$ clears the lowest set bit

2. Single Number — the classic XOR trick

Steps: XOR every element in the array together → explain why XOR-ing a number with itself cancels to 0 → extend to 'every element appears three times except one' using bit counting per position

3. Counting Bits

Steps: count set bits for every number from 0 to n using Integer.bitCount() first, as a baseline → then implement it via DP: $\text{bits}(i) = \text{bits}(i \gg 1) + (i \& 1)$ → compare both approaches' time complexity

4. Power of Two / Power of Four Check

Steps: check power-of-two using $n > 0 \&\& (n \& (n-1)) == 0$ → extend to power-of-four, accounting for which bit position is set → test with 0 and with negative input

5. Subsets via Bitmask — connecting bit manipulation back to Backtracking

Steps: represent each subset as an n -bit integer from 0 to $2^n - 1$ → for each integer, read off which elements are included by checking each bit → compare this bitmask approach to the recursive backtracking version from the Backtracking topic

Study Resources

- GeeksforGeeks — Bitwise Algorithms — <https://www.geeksforgeeks.org/dsa/bitwise-algorithms/>
- GeeksforGeeks — What is Bit Manipulation — <https://www.geeksforgeeks.org/dsa/what-is-bit-manipulation/>
- NeetCode — Bit Manipulation — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 6 problems (4 Easy / 2 Medium), on top of the hands-on builds above.

Dynamic Programming

Solving a big problem by breaking it into overlapping smaller subproblems and never solving the same one twice.

Subtopics: memoization vs. tabulation · house-robber-style linear DP · knapsack pattern · longest common subsequence · coin change · state-machine DP

Suggested Projects — Detailed Instructions

1. Fibonacci: Naive → Memoized → Tabulated

Steps: implement naive recursive Fibonacci and time it for $n=35$ → add memoization with a HashMap or array cache → rewrite it bottom-up with tabulation, no recursion → compare all three approaches' time and space

2. House Robber — the bridge from linear arrays into constrained-choice DP

Steps: define $dp[i]$ as the max take through house i , either skipping or robbing it → handle House Robber II where houses are arranged in a circle → trace back which houses were actually chosen, not just the max total

3. 0/1 Knapsack — the canonical DP pattern

Steps: given items with weight/value and a capacity, maximize value → solve with a 2D DP table (items x capacity) → trace back which items were chosen → test with a capacity that fits everything vs. almost nothing

4. Longest Common Subsequence

Steps: build a 2D DP table comparing two strings → return the length of the longest common subsequence → extend it to reconstruct the actual subsequence, not just its length → test with strings that share nothing in common

5. Coin Change (Unbounded Knapsack)

Steps: solve for the minimum number of coins to reach a target amount → handle the impossible case: return -1 → solve the count-the-ways variant as a follow-up → compare unbounded (reuse allowed) vs. 0/1 knapsack's constraint

6. DP on a Grid: Unique Paths

Steps: count the number of unique paths from top-left to bottom-right, moving only right/down → solve with a 2D DP table → add obstacles that block certain cells → confirm the obstacle version still produces correct counts

7. State-Machine DP: Buy/Sell Stock with Cooldown

Steps: define states explicitly: holding, sold-today (cooldown), and free-to-buy → write the transition equations between the three states day by day → extend it to include a transaction fee → compare this to the simpler single-transaction stock problem to see what state actually adds

Study Resources

- GeeksforGeeks — Dynamic Programming — <https://www.geeksforgeeks.org/dsa/dynamic-programming/>
- Baeldung — Dynamic Programming in Java — <https://www.baeldung.com/java-dynamic-programming>
- NeetCode — Dynamic Programming — <https://neetcode.io/roadmap>

Suggested LeetCode volume: 20 problems (3 Easy / 12 Medium / 5 Hard), on top of the hands-on builds above.

Segment Trees & Fenwick Trees

Correctly left out of the core loop by the original curriculum — most companies don't ask this outside HFT/quant or competitive-programming-heavy bars. Include only if that's your target.

Subtopics: segment tree build/query/update · lazy propagation (concept only) · Fenwick Tree (Binary Indexed Tree) for prefix sums

Suggested Projects — Detailed Instructions

1. Segment Tree for Range Sum Queries

Steps: build a segment tree over an array in $O(n)$ → implement a range-sum query in $O(\log n)$ → implement a point update in $O(\log n)$ → compare to the Prefix Sum approach from Arrays & Strings — explain when each is the right tool

2. Fenwick Tree (Binary Indexed Tree)

Steps: implement `update()` and `prefixSum()` using the BIT bit-trick indexing → compare code complexity and constant-factor speed to the segment tree version → test both structures against the same range-sum workload

Study Resources

- GeeksforGeeks — Segment Tree — <https://www.geeksforgeeks.org/dsa/segment-tree-data-structure/>
- GeeksforGeeks — Binary Indexed Tree — <https://www.geeksforgeeks.org/dsa/binary-indexed-tree-or-fenwick-tree-2/>

Suggested LeetCode volume: 4 problems (treat as optional — only if targeting quant/HFT/competitive programming), on top of the hands-on builds above.

Final Project for This Phase

DSA Inside ShopFlow — Practical Applications

Apply your DSA knowledge to real features in your existing ShopFlow project. This ties the abstract concepts to code you already understand — and now covers every new topic added in this revision, not just the original nine.

Product search	trie-based autocomplete for the search bar
Category tree	recursive tree traversal to build the nested category menu
Order queue	priority queue (heap) for order processing by urgency/SLA
Recommendations	graph of 'users who bought X also bought Y' (BFS)
Inventory restock	min-heap to surface lowest-stock items first
Delivery routing	Dijkstra on a city graph for shortest delivery path
Coupon/discount selection	greedy interval scheduling to maximize non-overlapping active promotions
Duplicate customer merge	union-find to merge accounts that share an email or phone number
Order combination generator	backtracking to generate valid bundle/combo offers under a price constraint
Feature flags & permission sets	bitmasking to store and check user roles/flags in a single int

Part D (Revised) complete. This version covers the CS fundamentals most self-taught developers skip, plus the pattern families — recursion, backtracking, greedy, bit manipulation, tries — that interviewers actually reach for. Next: Part E — Career & Interview Readiness.

Part D (Revised) — Progress Tracker

Check off each topic once you've read its concept sections, built all hands-on projects, worked through its suggested LeetCode set, and folded the result into ShopFlow where indicated.

■	9.0 Recursion & Recursive Thinking (NEW)	5 projects	8 LC problems
■	9.1 Arrays & Strings	6 projects	15 LC problems
■	9.2 Hashing	5 projects	10 LC problems
■	9.3 Two Pointers & Sliding Window — Pattern Review (NEW)	5 projects	8 LC problems
■	9.4 Stacks & Queues	6 projects	10 LC problems
■	9.5 Sorting & Searching	5 projects	10 LC problems
■	9.6 Linked Lists	6 projects	10 LC problems
■	9.7 Backtracking (NEW)	5 projects	10 LC problems
■	9.8 Trees	5 projects	15 LC problems
■	9.9 Heaps & Priority Queues	6 projects	8 LC problems
■	9.10 Tries (NEW)	5 projects	5 LC problems
■	9.11 Graphs	6 projects	15 LC problems
■	9.12 Union-Find (Disjoint Set) (ELEVATED)	4 projects	5 LC problems
■	9.13 Greedy Algorithms (NEW)	5 projects	8 LC problems
■	9.14 Bit Manipulation (NEW)	5 projects	6 LC problems
■	9.15 Dynamic Programming	7 projects	20 LC problems
■	9.16 Segment Trees & Fenwick Trees (OPTIONAL / ADVANCED)	2 projects	4 LC problems

Total: 17 topics (16 core + 1 optional) · 85 hands-on projects · ~163 suggested LeetCode problems · 10–13 weeks at 8–12 hrs/week.